AWS Cloud Developer
Interview & Concept Preparation Guide

EC2 | S3 | AWS CLI | IAM & Security | Automation & IaC
Basics to Advanced Concepts, Job-Aligned Q&A, and Tricky Scenario Questions

---

Prepared based on: Cloud Developer (AWS EC2, S3, CLI) job description

Table of Contents

1. How This Guide Maps to the Job Description

This job description centers on three AWS pillars — **EC2** (compute), **S3** (object storage), and **AWS CLI** (automation) — wrapped in enterprise concerns: security, cost optimization, monitoring, migrations, and hybrid-team collaboration. The table below maps each responsibility line to the knowledge area you should be ready to speak to.

JD Responsibility	Core Skill Area
Design secure, scalable EC2/S3 solutions	EC2 architecture, S3 design, Well-Architected Framework
Develop AWS CLI automation scripts	AWS CLI, shell scripting, JSON/JMESPath querying
Object storage lifecycle, durability, cost	S3 storage classes, lifecycle policies, versioning
EC2 sizing, AMIs, networking	Instance types, AMI management, VPC basics
Infrastructure as code patterns	CloudFormation/Terraform concepts, scripting standards
Monitor workloads, remediate bottlenecks	CloudWatch, logs, metrics, alarms
EC2 <-> S3 integration, data integrity	IAM roles, presigned URLs, VPC endpoints
Security best practices	IAM policies, encryption (KMS), security groups/NACLs
Rightsizing, cost control	Cost Explorer, Compute Optimizer, S3 storage class analysis
Root cause analysis on incidents	Structured troubleshooting, CloudTrail, log correlation
Documentation & runbooks	Operational documentation practices
Migration cutover planning	Data migration tools: DataSync, Snowball, S3 replication

2. AWS EC2 — Concepts, Basics to Advanced

2.1 What EC2 Is

Amazon EC2 (Elastic Compute Cloud) provides resizable virtual servers ("instances") in the AWS cloud. You choose an OS image (AMI), a hardware profile (instance type), storage, and networking, and AWS handles the underlying physical infrastructure. Billing is usage-based (per second/hour depending on purchase option).

2.2 Core Building Blocks

- **AMI (Amazon Machine Image):** A template containing OS, application server, and applications used to launch an instance.
- **Instance type:** Defines vCPU, memory, storage, and network capacity (e.g., t3.micro, m5.large, c6g.xlarge).
- **EBS (Elastic Block Store):** Persistent block storage volumes attached to instances; survives instance termination if configured to.
- **Instance store:** Temporary disk physically attached to the host; data is lost on stop/termination.
- **Security Group:** A virtual, stateful firewall attached to instances controlling inbound/outbound traffic.
- **Key Pair:** Public/private key used for SSH (Linux) or to decrypt the administrator password (Windows).
- **Elastic IP:** A static public IPv4 address you can allocate and remap between instances.
- **Placement Group:** Controls how instances are placed on underlying hardware (cluster, spread, partition) for performance or fault isolation.

2.3 Instance Lifecycle & States

- **Pending -> Running -> Stopping -> Stopped -> Terminated** are the primary states.
- **Stop vs Terminate:** Stopping preserves EBS-backed root volumes and can be restarted; terminating deletes the instance permanently (and its root EBS volume by default, unless 'Delete on Termination' is disabled).
- **Reboot** retains the same underlying host in most cases; **Stop/Start** can move the instance to new underlying hardware, which is why a Stop/Start is often used to resolve host-level hardware issues.
- **Hibernate** saves RAM contents to the EBS root volume so the instance resumes faster with in-memory state intact (Linux and some Windows AMIs, with prerequisites).

2.4 Instance Types & Sizing

Instance families are optimized for different workloads. Understanding the family letter helps you reason about sizing decisions quickly during interviews.

Family	Optimized For	Example Use Case
T (t3, t4g)	Burstable, low-cost general purpose	Dev/test, low-traffic web apps
M (m5, m6i)	Balanced compute/memory/network	General app servers, small-mid DBs

Family	Optimized For	Example Use Case
C (c5, c6g)	Compute-optimized (high vCPU:RAM)	Batch processing, video encoding, HPC
R (r5, r6g)	Memory-optimized	In-memory caches, large databases
I (i3, i4i)	High-speed local NVMe storage	NoSQL DBs, data warehousing
G/P (g5, p4)	GPU-accelerated	ML training/inference, graphics rendering

Rightsizing means matching instance size to actual CPU/memory/network utilization, typically guided by CloudWatch metrics or AWS Compute Optimizer, to avoid overpaying for idle capacity.

2.5 Purchasing Options

Option	Description	Best For
On-Demand	Pay per second/hour, no commitment	Unpredictable, short-term workloads
Reserved Instances	1 or 3-year commitment for discount (up to ~72%)	Steady-state, predictable workloads
Savings Plans	Commit to $/hour spend for 1-3 years, flexible across instance families	Predictable spend, flexible compute mix
Spot Instances	Bid on spare capacity at up to 90% discount; can be reclaimed	Fault-tolerant, batch, stateless workloads
Dedicated Hosts/Instances	Physical server dedicated to you	Compliance/licensing requirements

2.6 Storage for EC2

- **EBS volume types:** gp3/gp2 (general purpose SSD), io2/io1 (provisioned IOPS SSD for high-performance DBs), st1 (throughput HDD for big data), sc1 (cold HDD for infrequent access).
- **Snapshots:** Point-in-time, incremental backups of EBS volumes stored in S3 (managed by AWS, not directly browsable). Used for backup, disaster recovery, and creating new AMIs.
- **EBS is AZ-bound:** a volume only attaches to instances in the same Availability Zone; you must snapshot and restore into another AZ to move it.

2.7 Networking for EC2

- Each instance launches inside a **VPC** subnet (public or private).
- **Public subnet:** has a route to an Internet Gateway; instances can get a public IP.
- **Private subnet:** no direct internet route; outbound internet access (e.g., for patching) typically goes through a **NAT Gateway**.
- **Security Groups** are stateful and apply at the instance (ENI) level; **Network ACLs** are stateless and apply at the subnet level.
- An **Elastic Network Interface (ENI)** is a virtual network card; instances can have multiple ENIs for dual-homed networking.

2.8 Scaling & High Availability

- **Auto Scaling Groups (ASG):** automatically add/remove instances based on demand or schedules, using launch templates.
- **Elastic Load Balancer (ALB/NLB):** distributes traffic across instances in multiple AZs and performs health checks.
- Designing for high availability means spreading instances across at least two **Availability Zones**, not just multiple instances in one AZ.

2.9 Advanced EC2 Topics

- **User Data:** A bootstrap script that runs once at first launch (e.g., installing packages), key for automation and IaC patterns.
- **IAM Instance Profile / Role:** Attaches temporary, auto-rotated credentials to an instance so applications can call AWS APIs (e.g., S3) without hardcoded keys.
- **Instance Metadata Service (IMDS):** Local endpoint (169.254.169.254) instances query for metadata/credentials; IMDSv2 (token-based) is the security-hardened version that mitigates SSRF-based credential theft.
- **Golden AMI pipeline:** Building pre-baked, hardened, patched AMIs (often with EC2 Image Builder or Packer) so instances launch consistently and quickly.
- **Enhanced networking / Elastic Fabric Adapter (EFA):** For high-throughput, low-latency networking in HPC/ML clusters.
- **Termination protection & lifecycle hooks:** Prevent accidental termination and allow custom actions (e.g., draining connections) during ASG scale-in.

3. AWS S3 — Concepts, Basics to Advanced

3.1 What S3 Is

Amazon S3 (Simple Storage Service) is an object storage service. Data is stored as objects (data + metadata + key) inside buckets, not as files in a traditional filesystem. It offers virtually unlimited scale, 11 nines of durability (99.999999999%), and a flat namespace addressed by object keys.

3.2 Core Concepts

- **Bucket:** A globally unique-named container for objects, created in a specific AWS Region.
- **Object:** The file plus metadata, identified by a unique **key** (full path-like string) within a bucket.
- **Object size:** 0 bytes up to 5TB per object; single PUT supports up to 5GB, larger requires multipart upload.
- **Prefix:** The 'folder-like' portion of a key (e.g., logs/2026/07/) used for organization and filtering; S3 has no real directories.
- **Versioning:** When enabled, every write to a key creates a new version instead of overwriting, protecting against accidental deletes/overwrites.

3.3 Storage Classes

Storage Class	Use Case	Key Trait
S3 Standard	Frequently accessed data	Millisecond access, highest cost
S3 Intelligent-Tiering	Unknown/changing access patterns	Auto-moves objects between tiers
S3 Standard-IA	Infrequent access, needs millisecond retrieval	Lower storage cost, retrieval fee
S3 One Zone-IA	Infrequent, re-creatable data	Single AZ, lower cost, less resilient
S3 Glacier Instant Retrieval	Archive with millisecond access	Cheap storage, quarterly-ish access
S3 Glacier Flexible Retrieval	Archive, minutes-to-hours retrieval	Very low cost
S3 Glacier Deep Archive	Long-term archive (7-10+ yrs)	Lowest cost, 12-48 hr retrieval

3.4 Lifecycle Management

- **Lifecycle policies** automatically transition objects between storage classes (e.g., Standard -> IA after 30 days -> Glacier after 90 days) or expire (delete) them after a set time.
- Lifecycle rules can also clean up incomplete multipart uploads and expire old noncurrent versions when versioning is on, controlling storage cost creep.

3.5 Durability, Availability & Consistency

- S3 stores data redundantly across a minimum of three Availability Zones within a Region (except One Zone-IA).
- S3 now provides **strong read-after-write consistency** for all operations (PUTs and DELETEs are immediately reflected in subsequent reads/lists).
- Durability (11 nines) refers to not losing data; Availability (e.g., 99.9%) refers to the service being reachable, they are different SLAs.

3.6 Security & Access Control

- **Bucket Policy:** Resource-based JSON policy attached to the bucket, controlling who can do what.
- **IAM Policy:** Identity-based policy attached to users/roles; combined with bucket policy, the most restrictive applicable rule wins for cross-account context, but within the same account any explicit allow (absent an explicit deny) grants access.
- **Block Public Access:** Account/bucket-level setting that overrides ACLs/policies to prevent public exposure; a key security guardrail.
- **ACLs (Access Control Lists):** Legacy, object/bucket-level permission mechanism; AWS now recommends disabling ACLs and using policies instead.
- **Presigned URLs:** Time-limited URLs generated with a signing identity's credentials that grant temporary access to a private object without changing permissions.
- **Encryption:** SSE-S3 (AWS-managed keys), SSE-KMS (customer-managed keys in KMS, with audit trail via CloudTrail), SSE-C (customer-supplied keys), and client-side encryption before upload.
- **VPC Endpoint (Gateway type) for S3:** Lets EC2 instances in private subnets reach S3 without traversing the public internet or needing a NAT Gateway.

3.7 Data Protection & Governance

- **Versioning + MFA Delete:** Extra protection requiring MFA to permanently delete versions or change versioning state.
- **Object Lock (WORM):** Write-once-read-many protection for compliance (retention periods, legal holds) — cannot be deleted or overwritten even by the account root during the retention window.
- **Replication (CRR/SRR):** Cross-Region or Same-Region Replication automatically copies objects to another bucket for DR, latency, or compliance needs.
- **S3 Access Points:** Named network endpoints with their own policies, simplifying access management for shared buckets across many applications/teams.

3.8 Performance & Large-Scale Data

- **Multipart Upload:** Splits large objects into parts uploaded in parallel, improving throughput and allowing resume after failure; recommended above ~100MB, required above 5GB.
- **S3 Transfer Acceleration:** Uses CloudFront edge locations to speed up uploads over long distances.

- **Request rate:** S3 automatically scales to support very high request rates per prefix; historically people avoided sequential key naming (e.g., timestamps) to avoid hot partitions, though this is far less of a concern with modern S3.
- **S3 Select / Athena:** Query data directly inside objects (e.g., CSV/JSON/Parquet) without retrieving the whole object, cutting cost and latency for analytics.

3.9 Migration & Large-Scale Ingest Tools

- **AWS DataSync:** Automated, scheduled transfer of large datasets between on-prem storage and S3 (or between AWS storage services).
- **AWS Snowball / Snowball Edge:** Physical device for offline bulk data transfer when network transfer is too slow or costly.
- **S3 Batch Operations:** Run an action (copy, tag, restore, invoke Lambda) across billions of objects listed in a manifest.

4. AWS CLI — Concepts, Basics to Advanced

4.1 What the AWS CLI Is

The AWS Command Line Interface is a unified tool to manage AWS services from a terminal or script. It wraps the AWS APIs (the same ones the console and SDKs use), and is the backbone of most operational automation referenced in this job description.

4.2 Installation & Configuration Basics

- Install via package manager or official installer; check version with `aws --version`.
- `aws configure` sets Access Key ID, Secret Access Key, default region, and output format, stored in `~/.aws/credentials` and `~/.aws/config`.
- **Named profiles** (`aws configure --profile prod`) let you manage multiple accounts/roles; invoke with `--profile prod` or the `AWS_PROFILE` env variable.
- Output formats: `json` (default, scriptable), `text`, `table`, and `yaml`.

4.3 Command Structure

The general pattern is: `aws <service> <operation> [parameters]`. Examples:

- `aws ec2 describe-instances --instance-ids i-0abc123`
- `aws s3 ls s3://my-bucket/prefix/`
- `aws s3api create-bucket --bucket my-bucket --region us-east-1`

Note the two S3 command sets: **s3** (high-level, user-friendly: cp, sync, mv, rm, ls) and **s3api** (low-level, maps 1:1 to the REST API, exposing every parameter such as ACLs, lifecycle configuration, and encryption settings).

4.4 Filtering & Querying Output

- **--query** uses JMESPath to extract/filter fields from JSON responses, e.g. `--query 'Reservations[].Instances[].InstanceId'`.
- **--filters** asks the AWS API itself to filter server-side (more efficient than pulling everything and filtering client-side).
- Piping to `jq` is common for further JSON manipulation beyond JMESPath's capability.
- `--output text` combined with `--query` is a common pattern for feeding values into shell loops.

4.5 Common Operational Scripting Patterns

- **Bulk resource management:** looping over describe-* output to stop/start/tag many resources at once.
- **Provisioning automation:** scripting `run-instances`, `create-bucket`, `put-bucket-lifecycle-configuration` etc. as repeatable, version-controlled scripts.

- **Waiters:** `aws ec2 wait instance-running` blocks a script until a resource reaches a target state, useful in deployment pipelines.
- **Dry runs:** many EC2 commands support `--dry-run` to validate permissions/parameters without making the change.
- **Pagination:** the CLI auto-paginates by default; use `--no-paginate` or `--page-size`/`--max-items` plus `--starting-token` to control large result sets manually.

4.6 S3-Specific CLI Operations

Command	Purpose
aws s3 cp	Copy files to/from/within S3 (supports --recursive)
aws s3 sync	Sync a local dir / S3 prefix, only transferring changed objects
aws s3 rm --recursive	Delete objects under a prefix
aws s3api put-bucket-versioning	Enable/suspend versioning on a bucket
aws s3api put-bucket-lifecycle-configuration	Apply lifecycle rules (transitions/expirations)
aws s3api put-bucket-encryption	Set default SSE-S3/SSE-KMS encryption for a bucket
aws s3 presign	Generate a presigned URL for temporary object access

4.7 EC2-Specific CLI Operations

Command	Purpose
aws ec2 run-instances	Launch new instance(s) from an AMI
aws ec2 describe-instances	List/inspect instances and their attributes
aws ec2 stop-instances / start-instances	Stop/start instances
aws ec2 create-snapshot	Create a point-in-time EBS snapshot
aws ec2 create-image	Create a new AMI from a running/stopped instance
aws ec2 authorize-security-group-ingress	Add an inbound rule to a security group
aws ec2 describe-instance-status	Check instance and system status checks

4.8 Identity, Credentials & Security in the CLI

- **IAM users vs roles:** Long-lived access keys (users) should be avoided in favor of temporary credentials via **IAM roles** and `aws sts assume-role`.
- `aws sts get-caller-identity` is the fastest way to confirm which identity/account the CLI is currently authenticated as, essential when debugging permission errors.

- **MFA-protected API access** can be enforced via IAM policy conditions, requiring `sts get-session-token` with an MFA code before sensitive calls succeed.
- Credentials resolve in a specific **precedence order**: CLI options -> environment variables -> credentials file -> container/instance role (IMDS). Knowing this order is critical for debugging 'wrong account' surprises.

4.9 Advanced CLI Usage

- **aws configure sso** integrates the CLI with IAM Identity Center (AWS SSO) for short-lived, centrally managed credentials, common in enterprise environments.
- **Custom retry/timeout tuning** via `AWS_MAX_ATTEMPTS` and `AWS_RETRY_MODE=adaptive` for scripts hitting throttling under load.
- **CLI + CI/CD:** pipelines invoke the CLI (or SDKs) inside build steps to validate configuration drift, run `--dry-run` checks, or push deployment artifacts, exactly the 'AWS CLI based steps' referenced in the job description.
- **Cross-account automation:** assuming a role in another account (`sts assume-role`) to run centralized scripts across an AWS Organization.

5. IAM, Security & Networking Essentials

5.1 IAM Fundamentals

- **Users, Groups, Roles, Policies** are the four core IAM constructs. Policies are JSON documents with Effect/Action/Resource/Condition.
- **Principle of least privilege:** grant only the permissions needed, nothing more, reviewed regularly.
- **Roles vs Users:** Roles provide temporary credentials assumed by a trusted principal (an EC2 instance, another service, or a federated user); they are preferred over long-lived user access keys.
- **Policy evaluation logic:** an explicit Deny always wins; without an explicit Deny, an explicit Allow (from any attached policy) grants access; with no matching Allow, the request is implicitly denied.

5.2 Encryption

- **Encryption at rest:** EBS volume encryption, S3 SSE-S3/SSE-KMS, RDS encryption, all typically backed by AWS KMS.
- **Encryption in transit:** TLS for API calls and application traffic; enforced on S3 via bucket policy conditions like aws:SecureTransport.
- **KMS Customer Managed Keys (CMK)** give you control over key rotation, policy, and an audit trail (via CloudTrail) of every use, versus AWS-managed keys.

5.3 Network Segmentation

- **VPC:** An isolated virtual network; subdivided into public and private **subnets** across Availability Zones.
- **Security Groups (stateful, instance-level)** vs **NACLs (stateless, subnet-level)** together form defense in depth.
- **NAT Gateway:** Lets private-subnet instances reach the internet outbound (e.g., for patching) without being reachable inbound.
- **VPC Peering / Transit Gateway:** Connect VPCs to each other (or many-to-many) for shared services or hybrid architectures.
- **VPC Endpoints (Gateway/Interface):** Private connectivity to AWS services (like S3 or DynamoDB) without traversing the public internet.

5.4 Compliance & Governance Tools

- **AWS Config:** Tracks resource configuration changes and evaluates compliance against rules (e.g., 'no public S3 buckets').
- **AWS CloudTrail:** Logs every API call made in the account, essential for security auditing and root-cause analysis.
- **AWS Trusted Advisor / Security Hub:** Aggregate security, cost, and performance recommendations.

6. Monitoring, Troubleshooting & Cost Optimization

6.1 Monitoring Stack

- **Amazon CloudWatch Metrics:** Time-series data (CPUUtilization, NetworkIn/Out, StatusCheckFailed for EC2; BucketSizeBytes, NumberOfObjects, 4xx/5xxErrors for S3).
- **CloudWatch Logs:** Centralized log aggregation (application logs, VPC Flow Logs, S3 access/server logs shipped via CloudTrail data events).
- **CloudWatch Alarms:** Trigger notifications or automated actions (e.g., SNS notification, Auto Scaling action) when a metric crosses a threshold.
- **CloudWatch Dashboards:** Custom visual views combining multiple metrics/services for operational visibility.
- **VPC Flow Logs:** Capture IP traffic metadata for a network interface, useful for diagnosing connectivity/security issues.

6.2 Structured Root Cause Analysis

- 1. **Define the symptom precisely** (what, when, scope, impact) before touching anything.
- 2. **Check recent changes first** (deployments, config changes, IAM changes via CloudTrail) as the most common root cause.
- 3. **Correlate metrics and logs** across the stack (EC2 CPU/network, ALB 5xx, S3 error rates) to localize the failing component.
- 4. **Reproduce/isolate** in a safe scope before applying a fix broadly.
- 5. **Apply a fix, verify recovery against metrics**, then document the incident and a preventive action (the 'preventive improvements' called out in the job description).

6.3 Cost Optimization Levers

- **Rightsizing** EC2 instances and EBS volumes based on actual utilization (Compute Optimizer, CloudWatch).
- **Purchasing options:** shifting steady workloads to Reserved Instances/Savings Plans, and fault-tolerant batch jobs to Spot.
- **S3 storage class transitions** and Intelligent-Tiering to stop paying Standard rates for cold data.
- **Cleaning up orphaned resources:** unattached EBS volumes, old snapshots, unused Elastic IPs, idle load balancers.
- **AWS Cost Explorer & Budgets:** visualize spend trends and set proactive alerts before overspend happens.

7. Infrastructure as Code, Automation & CI/CD Context

7.1 Why IaC Matters Here

The job description asks for 'infrastructure as code patterns using scripting and configuration tools' — this can mean anything from well-structured AWS CLI/Bash scripts to dedicated IaC tools. Understanding the spectrum shows maturity.

7.2 The IaC Spectrum

Approach	Example	Trade-off
Manual console clicks	AWS Console	Fast but not repeatable, error-prone, no history
Scripted CLI automation	Bash + AWS CLI	Repeatable, simple, but limited state tracking
Declarative IaC	CloudFormation, Terraform, CDK	Full state management, drift detection, repeatable
Configuration management	Ansible, Chef, Puppet	Good for in-instance configuration/patching

7.3 Key Concepts to Know

- **Idempotency:** Running a script/template multiple times produces the same end state without duplicating resources, a property good automation scripts should have.
- **Drift detection:** Identifying when real infrastructure has diverged from the IaC definition (e.g., a manual console change).
- **State management:** Terraform's state file or CloudFormation's stack state tracks what exists so the tool knows what to create/update/delete.
- **CI/CD integration:** Pipelines (CodePipeline/CodeBuild, GitHub Actions, Jenkins) that run `aws cloudformation deploy`/`terraform apply` or validation CLI calls as an automated gate before release.
- **Containerization context:** Even in an EC2/S3-centric role, familiarity with Docker/ECS/EKS is a common complementary skill mentioned as a plus in this JD.

8. Interview Q&A; — EC2

Q1. What is Amazon EC2 and how does it differ from traditional on-premises servers?

A: EC2 provides virtual, resizable compute capacity in the cloud, billed on usage rather than owned as capital hardware. Unlike on-prem servers, you can provision, resize, or terminate capacity in minutes, and AWS handles the physical hardware, power, and data-center operations.

Q2. What is the difference between stopping and terminating an instance?

A: Stopping shuts down the instance but preserves its EBS-backed root volume so it can be restarted later; you stop paying for compute but still pay for the attached EBS storage. Terminating permanently deletes the instance and, by default, its root EBS volume (unless Delete on Termination is disabled).

Q3. How do you choose the right EC2 instance type for a workload?

A: Start from the workload's bottleneck: CPU-bound batch jobs suit compute-optimized (C-family); memory-heavy databases suit R-family; balanced web/app tiers suit M-family; bursty, low-traffic workloads suit T-family. Validate the choice with actual CloudWatch metrics or Compute Optimizer recommendations rather than guessing.

Q4. What is the difference between EBS and instance store volumes?

A: EBS is persistent, network-attached block storage that survives instance stop/termination (if configured) and can be detached/reattached. Instance store is physically attached to the host, offers very high IOPS, but is ephemeral, all data is lost on stop or termination.

Q5. What is an AMI and why would you build a custom one?

A: An AMI is a template (OS + configured software + settings) used to launch instances. Custom 'golden' AMIs let you pre-bake security hardening, patches, and application dependencies so new instances launch faster and more consistently than running full configuration at boot.

Q6. How do Security Groups differ from Network ACLs?

A: Security Groups are stateful and applied at the instance/ENI level, if you allow inbound traffic, the response is automatically allowed out. NACLs are stateless and applied at the subnet level, you must explicitly allow both directions, and rules are evaluated in numbered order.

Q7. How would you give an EC2 instance access to an S3 bucket securely?

A: Attach an IAM role (instance profile) to the instance with a policy scoped to the specific bucket and required actions, rather than embedding access keys on the instance. The EC2 metadata service supplies short-lived, automatically rotated credentials to the SDK/CLI running on the instance.

Q8. What is User Data used for?

A: A script (bash, cloud-init, PowerShell) that runs once when an instance first launches, commonly used to install packages, pull configuration, or register the instance with a service, a key building block for automated, repeatable provisioning.

Q9. How do you design EC2 for high availability?

A: Deploy instances across at least two Availability Zones behind an Elastic Load Balancer, use an Auto Scaling Group with health checks to replace unhealthy instances automatically, and store session/state outside the instance (e.g., in S3, DynamoDB, or ElastiCache) so any instance can serve any request.

Q10. What's the difference between On-Demand, Reserved, Savings Plans, and Spot instances?

A: On-Demand has no commitment and the highest per-hour price. Reserved Instances and Savings Plans trade a 1-3 year commitment for a significant discount on steady-state usage. Spot instances use spare AWS capacity at up to ~90% discount but can be reclaimed with short notice, so they suit fault-tolerant or interruptible workloads.

Q11. How would you troubleshoot an EC2 instance that suddenly became unreachable?

A: Check the instance status checks (system vs instance status) in the console/CLI, review CloudWatch CPU/network metrics for saturation, check security group and NACL rules for recent changes, and review VPC route tables. If system status check fails, a stop/start often migrates it to healthy underlying hardware; if instance status check fails, it's usually an OS-level issue requiring console log inspection.

Q12. What is IMDSv2 and why does it matter for security?

A: IMDSv2 is the token-based, session-oriented version of the EC2 Instance Metadata Service. Unlike IMDSv1's simple GET requests, it requires a PUT-based token first, which mitigates a class of server-side request forgery (SSRF) attacks that could otherwise be used to steal an instance's IAM role credentials.

9. Interview Q&A; — S3

Q1. What is Amazon S3 and what problem does it solve?

A: S3 is a highly durable, highly scalable object storage service, storing data as key/value objects in buckets rather than as files in a hierarchical filesystem. It removes the need to manage storage capacity, replication, or hardware, and is commonly used for backups, static assets, data lakes, and application storage.

Q2. What is the difference between S3 durability and availability?

A: Durability (99.999999999%, 11 nines) describes the probability that stored data is not lost, achieved through automatic replication across multiple Availability Zones. Availability (e.g., 99.9%) describes how reliably the service can be reached and respond to requests, a separate SLA.

Q3. How do S3 storage classes differ and how would you choose between them?

A: They trade storage cost against retrieval cost/latency: Standard for frequently accessed hot data, Standard-IA/One Zone-IA for infrequently accessed but instantly retrievable data, Intelligent-Tiering when access patterns are unpredictable, and the Glacier tiers for archival data accessed rarely, where minutes-to-hours retrieval time is acceptable in exchange for very low storage cost.

Q4. What is an S3 lifecycle policy and why is it important for cost management?

A: A lifecycle policy is a set of rules that automatically transitions objects to cheaper storage classes over time or expires (deletes) them after a defined period, so cost scales with actual value of the data rather than everything staying on expensive Standard storage indefinitely.

Q5. What is S3 versioning and when would you enable it?

A: Versioning keeps every previous version of an object instead of overwriting it on each PUT, protecting against accidental deletes and overwrites. It's commonly enabled for critical data buckets, though it does increase storage cost and requires lifecycle rules to clean up old noncurrent versions.

Q6. How does S3 encryption work, and what's the difference between SSE-S3 and SSE-KMS?

A: Both encrypt objects at rest using AES-256. SSE-S3 uses keys fully managed by AWS with no additional configuration. SSE-KMS uses keys managed in AWS KMS, giving you key rotation control, custom key policies, and a CloudTrail audit trail of every encryption/decryption call, at the cost of a small additional KMS API charge.

Q7. What is a presigned URL and when would you use one?

A: A presigned URL is a time-limited link generated using an IAM principal's credentials that grants temporary access (upload or download) to a private S3 object without changing the bucket's or object's permissions. It's commonly used to let a browser or third party upload/download directly to S3 without exposing AWS credentials.

Q8. How would you secure an S3 bucket that must never be publicly accessible?

A: Enable S3 Block Public Access at both the bucket and account level, avoid public bucket policies and ACLs, use IAM roles/bucket policies scoped to specific principals, enforce encryption in transit (aws:SecureTransport condition), and monitor with AWS Config rules that flag any public exposure.

Q9. What is multipart upload and why does it matter for large objects?

A: Multipart upload splits a large object into independently uploaded parts (in parallel), improving throughput and letting you resume just the failed parts instead of re-uploading the whole object. It's required for objects over 5GB and recommended well before that for reliability on unstable networks.

Q10. What is the difference between S3 bucket policies and IAM policies?

A: Both are JSON policy documents, but a bucket policy is resource-based and attached directly to the bucket (so it can grant access to other accounts), while an IAM policy is identity-based and attached to a user, group, or role. Access is granted if either allows it and neither explicitly denies it (for same-account access).

Q11. How would you plan a large-scale on-prem to S3 data migration?

A: Assess data volume and available bandwidth to choose the transfer method (DataSync/CLI sync for networked transfer, Snowball for very large offline transfers), run an initial bulk copy, then use incremental syncs to catch changes, validate data integrity (checksums/object counts) before cutover, and plan a brief cutover window to reconcile any final delta.

Q12. What are S3 Access Points and why would you use them over a single bucket policy?

A: Access Points are named endpoints, each with its own policy, that sit in front of a shared bucket. They let you decompose one complex bucket policy into simpler, per-application or per-team policies, which is easier to manage and audit at scale than one large policy on the bucket itself.

10. Interview Q&A; — AWS CLI & Automation

Q1. What is the AWS CLI and how does it relate to the AWS SDKs and Console?

A: All three are clients of the same underlying AWS APIs. The CLI is a scriptable command-line client ideal for automation and repeatable operations; SDKs let applications call AWS programmatically; the Console is the human-facing GUI. Anything doable in the Console can generally be scripted with the CLI, which is what makes it valuable for automation-heavy roles.

Q2. How do you manage multiple AWS accounts or environments with the CLI?

A: Using named profiles in ~/.aws/config and ~/.aws/credentials, invoked with --profile or the AWS_PROFILE environment variable. In enterprise setups this is often combined with aws configure sso or role assumption (sts assume-role) rather than storing long-lived static keys per environment.

Q3. What's the difference between the 'aws s3' and 'aws s3api' command sets?

A: 'aws s3' provides simplified, high-level commands (cp, sync, ls, rm) optimized for common file operations. 'aws s3api' exposes the full underlying REST API 1:1, needed when you require granular control, such as setting a specific ACL, lifecycle configuration, or object metadata that the high-level commands don't expose.

Q4. How would you extract just the instance IDs of running instances using the CLI?

A: Using --query with JMESPath, filtering server-side with --filters, and --output text for script-friendly output, for example: aws ec2 describe-instances --filters "Name=instance-state-name,Values=running" --query "Reservations[].Instances[].InstanceId" --output text.

Q5. How do you handle credentials securely in an automation script?

A: Avoid hardcoding access keys in scripts. On EC2, rely on an attached IAM role so the CLI picks up temporary credentials automatically from the instance metadata service. In CI/CD, use short-lived credentials via OIDC federation or role assumption rather than long-lived secrets stored in the pipeline.

Q6. What is a CLI 'waiter' and why is it useful in a deployment script?

A: A waiter (e.g., aws ec2 wait instance-running) polls the API until a resource reaches a target state, letting a script block until, for example, a newly launched instance is actually ready, instead of guessing with a fixed sleep delay.

Q7. How do you make an automation script idempotent?

A: Check current state before acting (e.g., describe-* to see if the resource already exists in the desired configuration) and only make the change if needed, or use tools/commands designed to be idempotent (like put-bucket-lifecycle-configuration, which fully replaces the config rather than appending). This avoids duplicate resources or errors on re-runs.

Q8. How would you bulk-tag or bulk-stop dozens of EC2 instances from the CLI?

A: Use describe-instances with --filters/--query to build a list of target instance IDs, then loop over that list (or pass the whole list at once, since many EC2 commands accept multiple IDs) to call create-tags or stop-instances, wrapping the script with logging and error handling for partial failures.

Q9. How do you debug 'Access Denied' errors when running a CLI command?

A: First confirm identity with aws sts get-caller-identity to rule out using the wrong account/role. Then check the relevant IAM policy (and, for S3, the bucket policy) for the exact action and resource ARN being called, watch for an explicit Deny anywhere in the policy chain, and check for conditions like MFA or source IP restrictions that may not be met.

Q10. How does the AWS CLI fit into a CI/CD pipeline?

A: Pipeline steps can invoke the CLI to validate configuration (dry-run checks), deploy infrastructure templates, push build artifacts to S3, or trigger deployments, acting as the glue between the build system and AWS resources, often using a pipeline-specific IAM role rather than static credentials.

11. Interview Q&A; — Security, Networking & IAM

Q1. What is the principle of least privilege and how do you apply it in IAM?

A: Grant only the specific actions and resources a principal needs to do its job, nothing broader. In practice this means scoping IAM policies to specific ARNs and actions instead of using wildcard resources, and preferring roles with narrowly-defined trust and permission policies over broad, standing user permissions.

Q2. Explain the difference between an IAM user, group, role, and policy.

A: A user is a persistent identity with long-term credentials; a group is a collection of users sharing permissions; a role is an identity assumed temporarily by a trusted principal (a service, another account, or a federated user) without permanent credentials; a policy is the JSON document that actually defines the permissions attached to any of the above.

Q3. How does IAM policy evaluation logic work when multiple policies apply?

A: AWS evaluates all applicable policies together: if any policy contains an explicit Deny, the request is denied regardless of other Allows. If there's no explicit Deny, the request is allowed if at least one policy explicitly allows it. If nothing allows it, the default is an implicit deny.

Q4. What's the difference between encryption at rest and in transit, and how is each achieved on AWS?

A: Encryption at rest protects stored data (e.g., EBS volume encryption, S3 SSE), typically backed by AWS KMS. Encryption in transit protects data moving across the network, achieved via TLS for API calls and application traffic, and can be enforced with policy conditions such as aws:SecureTransport on S3.

Q5. How would you design network segmentation for a 3-tier application in a VPC?

A: Public subnets host only the load balancer/bastion with routes to an Internet Gateway; application instances sit in private subnets reachable only from the load balancer's security group; database instances sit in even more restricted private subnets reachable only from the app tier's security group. Security groups and NACLs together enforce that only the intended tier-to-tier traffic is allowed.

Q6. What is a VPC endpoint and why would you use one for S3 access?

A: A VPC endpoint provides private connectivity from resources inside a VPC to an AWS service without traversing the public internet or requiring a NAT Gateway. For S3, a Gateway endpoint avoids NAT Gateway data-processing charges and keeps traffic on the AWS network, which is both a cost and a security improvement.

Q7. How do you audit who did what in an AWS account?

A: AWS CloudTrail logs every API call (who, what action, when, from where), which is the primary source for security audits and incident root-cause analysis. Combined with AWS Config for configuration history and CloudWatch Logs for application-level detail, you get end-to-end traceability.

12. Interview Q&A; — Monitoring, Cost & Troubleshooting

Q1. What CloudWatch metrics would you watch for an EC2-based web application?

A: CPUUtilization and StatusCheckFailed at the instance level, NetworkIn/NetworkOut and disk I/O for resource saturation, plus ALB-level metrics like RequestCount, TargetResponseTime, and HTTPCode_Target_5XX_Count to see application-facing health rather than just infrastructure health.

Q2. Walk through your approach to root-causing a production incident.

A: Define the symptom and its scope precisely, check what changed recently (deployments, config, IAM) as the most likely cause, correlate CloudWatch metrics and logs across the stack to localize the failing component, reproduce or isolate the issue safely, apply a fix, confirm recovery against metrics, and document the root cause plus a preventive action.

Q3. How would you identify and reduce EC2 cost without hurting performance?

A: Use CloudWatch/Compute Optimizer data to rightsize instances that are consistently underutilized, shift steady-state workloads to Reserved Instances or Savings Plans, move fault-tolerant/batch workloads to Spot, and clean up orphaned resources like unattached EBS volumes, old snapshots, and unused Elastic IPs.

Q4. How would you reduce S3 storage cost for a large, growing dataset?

A: Analyze access patterns with S3 Storage Lens or Intelligent-Tiering, apply lifecycle rules to move cold data to IA or Glacier tiers automatically, expire genuinely unneeded data, and clean up incomplete multipart uploads and stale noncurrent versions if versioning is enabled.

Q5. How do you set up proactive alerting so you find out about problems before users do?

A: Define CloudWatch Alarms on leading indicators (elevated error rates, latency, CPU/memory saturation, queue depth) rather than only lagging indicators, route alarms to SNS/chat integrations for on-call visibility, and pair with synthetic monitoring/health checks that test the user-facing path directly.

Q6. What's the difference between a metric-based alarm and a log-based investigation?

A: A metric-based alarm tells you something crossed a threshold (e.g., 5xx rate > 5%), which is good for fast detection, but it doesn't explain why. Log-based investigation (CloudWatch Logs Insights, correlating request IDs across services) is where you find the actual root cause once an alarm has told you where to look.

13. Interview Q&A; — Behavioral / Hybrid Team & Documentation

Q1. How do you document a cloud architecture so it's genuinely reusable by other teams?

A: Combine a high-level diagram (components, data flow, trust boundaries) with a concise text explanation of the 'why' behind key decisions, an operational runbook for common tasks/incidents, and versioned, source-controlled configuration so the doc doesn't drift from what's actually deployed.

Q2. Describe how you'd collaborate with an application team designing EC2-to-S3 integration.

A: Clarify the data contract first (object naming, format, expected volume/frequency), agree on the access pattern (direct SDK calls vs presigned URLs), scope an IAM role to only what's needed, and review the design together against durability, cost, and failure-handling requirements before building.

Q3. How do you stay effective working in a hybrid, day-shift, distributed team model?

A: Favor asynchronous, written communication for anything non-urgent (clear tickets, documented decisions) so distributed teammates aren't blocked waiting for a live conversation, and reserve synchronous time for design discussions or ambiguous problems that benefit from real-time back-and-forth.

Q4. Tell me about a time you had to troubleshoot a production issue under pressure.

A: A strong answer follows a structured arc: the symptom and business impact, how you triaged and narrowed the cause using metrics/logs, the fix applied, how you verified recovery, and the concrete follow-up action taken to prevent recurrence, showing process rather than luck.

14. Tricky & Scenario-Based Questions (with Traps Explained)

These questions are designed to catch surface-level knowledge. Each answer also explains the 'trap' the question is testing for.

T1. You stop and start an EC2 instance and its public IP address changes, but a colleague says 'reboot' didn't change it last week. Why the difference?

A: A standard public IP is released when an instance stops and a new one is assigned on start, because Stop/Start can move the instance to different underlying hardware. A reboot keeps the instance on the same host and doesn't release the network attachment, so the public IP stays the same. Trap: candidates often assume all restarts behave identically; the fix for a fixed IP is an Elastic IP, not just remembering to 'reboot instead of stop.'

T2. A teammate says S3 is 'eventually consistent' so you always need retry logic after a write. Is that still accurate?

A: No, this is outdated. S3 now provides strong read-after-write consistency for all operations, including overwrite PUTs and DELETEs; a read immediately after a write reflects that write. Trap: this is legacy knowledge many candidates repeat without realizing AWS changed this behavior; stating the old caveat unprompted signals stale knowledge.

T3. You terminate an EC2 instance but its EBS volume still shows up and is still being billed. What happened?

A: The 'Delete on Termination' flag on that volume (usually set per-device at launch) was disabled, so the root or attached volume was intentionally preserved rather than deleted. Trap: candidates assume termination always deletes attached storage; in reality this is configurable per volume, and non-root volumes default to NOT being deleted unless explicitly configured.

T4. Your S3 bucket policy grants public read access, but objects still return Access Denied to anonymous users. What's the most likely cause?

A: S3 Block Public Access is almost certainly enabled at the bucket or account level, which overrides bucket policies and ACLs that would otherwise grant public access. Trap: candidates dive into rewriting the bucket policy JSON when the real blocker is a separate account-level guardrail.

T5. Two IAM policies apply to a user: one explicitly denies s3:DeleteObject, the other explicitly allows it via a group membership. What happens when the user tries to delete an object?

A: The delete is denied. In AWS IAM evaluation logic, an explicit Deny anywhere in any applicable policy always overrides any Allow, regardless of how the Allow was granted. Trap: candidates who don't know the evaluation order assume the more specific or more recently attached policy wins.

T6. You need an EC2 instance in a private subnet to download OS patches from the internet, but security policy forbids giving it a public IP or an Elastic IP. How do you solve it?

A: Route the private subnet's outbound traffic through a NAT Gateway (or NAT instance) in a public subnet; the instance itself never needs a public IP because the NAT Gateway handles the internet-facing connection on its behalf. Trap: candidates conflate 'needs internet access' with 'needs a public IP,' missing that NAT provides outbound-only access without exposing the instance.

T7. A script using 'aws s3 sync' to a bucket appears to succeed with no errors, but a critical file with the wrong extension didn't get uploaded. Why might that happen, and how do you catch it?

A: sync only transfers files it thinks are new or changed based on size/timestamp comparisons, and it silently skips based on any exclude filters configured in the command; it won't error just because a file 'looks odd'. The real risk is usually an --exclude/--include pattern mismatch. Trap: candidates assume sync exhaustively verifies content; the fix is validating object counts or checksums after the sync, not just trusting exit code 0.

T8. You enable S3 Object Lock in Compliance mode on a bucket with a 1-year retention. A senior engineer insists they can still delete a locked object as the account root user in an emergency. True or false?

A: False. Object Lock in Compliance mode prevents deletion or overwrite by any user, including the root user, until the retention period expires; only Governance mode allows users with special permission (s3:BypassGovernanceRetention) to override it. Trap: this question tests whether candidates understand that Compliance mode is intentionally irreversible, even for AWS support.

T9. An Auto Scaling Group scales in and terminates an instance mid-request, causing dropped connections. What EC2/ASG feature specifically addresses this?

A: Connection draining/deregistration delay on the load balancer target group, combined with an ASG lifecycle hook (or simply allowing the deregistration delay to elapse) so in-flight requests finish before the instance is terminated. Trap: candidates jump straight to 'add more instances' when the actual gap is in the scale-in/termination sequencing, not capacity.

T10. Your team stores S3 access keys as environment variables on an EC2 instance for an application to use. What's wrong with this approach even if the keys are never leaked?

A: It bypasses IAM roles entirely, meaning the credentials are long-lived, not automatically rotated, and not scoped/auditable the way role-based temporary credentials are; they also have to be manually distributed and rotated, and the practice contradicts the AWS security best practice of never hardcoding long-term credentials on an instance. Trap: 'no leak occurred' isn't the bar; the architecture itself is the finding, not just the outcome.

T11. A colleague argues Reserved Instances are always cheaper than Spot Instances for batch jobs, so the team should switch. Do you agree?

A: Not necessarily. Spot can be up to ~90% cheaper than On-Demand and often cheaper than Reserved pricing too, but it can be interrupted with short notice, so it's only a good fit if the batch job is fault-tolerant/checkpointable. Trap: cost comparisons in isolation ignore the workload's tolerance for interruption; the 'right' answer depends on workload characteristics, not price alone.

T12. You run 'aws s3 ls' and it returns nothing for a bucket you know has objects, with no error message. What are the most likely causes to check, in order?

A: First, confirm you're pointed at the right bucket/prefix and region (some tools default to us-east-1). Second, run aws sts get-caller-identity to confirm you're using the expected identity/account. Third, check whether the identity has s3:ListBucket permission specifically (distinct from s3:GetObject), a common gap since listing and reading are separate permissions. Trap: candidates assume a permissions issue always throws an explicit error; a silent empty result is a classic sign of a ListBucket-vs-GetObject permission mismatch.

T13. Why might CloudWatch show an EC2 instance's CPUUtilization at a healthy 30%, while the application is still timing out under load?

A: CPU is only one resource; the bottleneck could be memory (which basic CloudWatch doesn't track without the CloudWatch agent), disk I/O/EBS throughput limits, network bandwidth, database connections, or an application-level thread/connection pool limit. Trap: candidates equate 'low CPU' with 'no resource problem,' missing that default EC2 metrics don't cover memory at all unless the CloudWatch agent is installed.

T14. A bucket has versioning enabled and someone runs 'aws s3 rm' on an object. Is the data gone?

A: No, a plain DELETE on a versioned bucket just adds a delete marker as the new 'current' version; the previous version(s) of the object still exist and can be restored by removing the delete marker or referencing the prior version ID. To permanently remove it you must delete that specific version ID. Trap: candidates think versioning is only about accidental overwrites, missing that deletes are also reversible under versioning.

T15. You're asked to make a migration cutover to S3 'zero-downtime.' What's the flaw in promising that literally?

A: Any cutover involving DNS changes, application redeploys, or final data reconciliation has some risk window; 'zero-downtime' is really 'minimized and tested downtime' achieved via incremental sync ahead of time, a short final delta sync, validation checksums, and a fast rollback plan. Trap: this tests whether a candidate overpromises; the stronger answer reframes the goal as risk minimization with a defined, tested rollback rather than an absolute guarantee.

15. Quick-Reference Cheat Sheet

EC2 Essentials

Concept	One-Line Reminder
Stop vs Terminate	Stop keeps EBS root volume; Terminate deletes it by default
Security Group	Stateful, instance-level firewall
NACL	Stateless, subnet-level firewall, ordered rules
IAM Instance Role	Preferred over hardcoded access keys on instances
IMDSv2	Token-based metadata access, mitigates SSRF credential theft
Placement Group	Cluster (perf), Spread (isolation), Partition (large distributed apps)

S3 Essentials

Concept	One-Line Reminder
Consistency	Strong read-after-write for all operations
Durability	11 nines, replicated across 3+ AZs (except One Zone-IA)
Versioning + delete	Delete adds a marker; data still recoverable
Object Lock Compliance	Cannot be deleted even by root until retention expires
Block Public Access	Overrides bucket policy/ACL grants of public access
Multipart upload	Required over 5GB, recommended above ~100MB

AWS CLI Essentials

Concept	One-Line Reminder
aws sts get-caller-identity	First command to run when access looks wrong
--query	JMESPath filtering of JSON response, client-side
--filters	Server-side filtering, more efficient at scale
Named profiles	Manage multiple accounts/roles via --profile
aws s3 vs aws s3api	High-level convenience vs full low-level API control
Credential precedence	CLI flag > env var > credentials file > instance role

End of guide. Review the tricky questions section last, once the fundamentals in Sections 2-7 feel comfortable, since most tricky questions are just a fundamental concept applied to an unexpected scenario.